

Sentinel - Data Flow Diagram

Neural Nexus · Smart India Hackathon 2026

Level 0 - System context

Sentinel is one system with four external entities. The **user** supplies code, questions, and budget figures. **GitHub** supplies repository contents. The **AI cloud** returns reasoning and embeddings. The **sandbox** provides an isolated execution room. Outputs are dashboards, financial exposure, fix plans, compliance scores, and a PDF report. Inside sit three stores: scan artifacts, risk state, and the document index.

Level 1 - Six processes

P1. Intake. Links, archives, or folders in; validated, fetched to an isolated per-scan copy, measured, and registered with a scan ID.

P2. Scanning. Agent crew attacks only the copy, or static patterns cover the same families; only proven hits with file, line, and proof are kept.

P3. Risk quantification. Damage (asset share, breach, downtime, fines, brand) times yearly chance (severity, score, exploitability, asset weight, trends) gives expected loss; simulation gives the worst case; scores follow.

P4. AI reasoning. Answers, ranked fixes, forecasts, and cited doc responses; poisoned chunks are screened out before inference.

P5. Investment and compliance. Best fix set per budget with returns and curve; every finding scored against five rulebooks.

P6. Presentation. Four screens, simulator, attack demo, PDF; last real result cached, cleared when backend is empty.

Level 2 - Process internals

P1: validate shape, token, archive type -> fetch (clone / extract / tree-write) -> measure language and lines -> register card. Bad input returns a plain error, stores nothing.

P2: gate tools (engine, sandbox, key) -> attack or pattern-match -> normalize to one shape -> persist report. Display cards are shaped at read time.

P3: damage per bug -> chance per bug -> asset merge -> 5,000-year simulation -> scores and history. What-ifs replay without touching stored data.

P4: embed question -> retrieve top chunks -> screen poison -> prompt with clean context plus live numbers -> cited answer.

P5: price fixes -> knapsack pack -> curve points -> clause map -> governance average.

Data stores

D1 scan copies plus per-scan reports, on disk. **D2** live findings, project card, bounded history, in memory.

D3 chunk vectors plus metadata, on disk. **D4** last verified result, in browser, auto-cleared on empty backend.

Trust boundary

Scanners never contact production or the open internet beyond the isolated copy. Single-use tokens pass once and vanish. Secrets stay in server-side configuration.